The Vertical Layer Problem

Josh · Secure Sovereign

Published August 27, 2026 · Updated October 30, 2026

The consensus layer is horizontal. The software that implements it is not.

Contents

- Benevolent dictatorship is right for ordinary software
- The hierarchy arrives as workplace habit
- Bitcoin produces agreement, not a product
- Refusing a release only delays the next download
- Undeclared concentration in the merge log
- Succession needs agreement that does not exist here
- Which thing gets forked
- What a horizontal development layer looks like
- The missing specification is doing the work
- Where this design fails
- The two questions that decide it
- Sources

Bitcoin cannot be governed like Linux. A dictator is a good way to ship a kernel, and a bad way to write Bitcoin's node software, even when the people who write it are competent and the code is good.

Bitcoin's consensus layer is horizontal. Every node checks blocks on its own, and no node can force another to take a bad one. The software that implements those rules is written by a hierarchy, and replacing the people who can merge, or making those people more accountable, leaves the hierarchy in place.

Operators download what that hierarchy ships. The money rules sit in the same program as the defaults, so a fight about who writes the client, or about what it ships by default, gets treated as a fight about Bitcoin. The protest people already know is a chain fork. You leave the chain by changing the money rules, which has happened many times, and each time it is a new coin. You stay on it by running a different program that still implements those rules.

A fight about the default client should not mean leaving the chain. A Core-derived fork can leave Core's defaults and stay on Bitcoin, and it is still mostly Core's code. An independently written client, checked against a written specification rather than Core's undocumented behavior, is the missing exit.

Nodes stay equal even though the people who write the code never will, and that still does not entitle those people to a throne.

Benevolent dictatorship is right for ordinary software

Benevolent dictatorship produces good software. Linux runs most of the internet's infrastructure and was governed for decades by one person's judgment about what merged. Python, Django, and Rust all had stretches of single-authority governance, and the work held up.

Concentrated authority makes architectural decisions faster than committees do. It keeps one design across a large codebase, and it ends fights that would otherwise eat years. Linux is a product, and a small group decides what goes in the kernel while everyone else downloads the result. Someone has to pick what ships, and a hierarchy is the right way to run that.

Forking the kernel is allowed. Almost nobody does it. The fork is yesterday's code. The patches people want keep going into the original project. Bitcoin node operators do the same thing. They download Core and wait for the next release. The difference is that Linux is only a product, while a Core release carries the wallet, the relay, and the money rules together, which is why an argument about the first two arrives as an argument about the third.

The pattern is right for ordinary software and wrong for money.

The hierarchy arrives as workplace habit

The hierarchy also shows up because paid engineers bring their workplace with them.

Most of the work is done by people employed to do it. The majority of Linux kernel contribution comes from paid engineers at Intel, Red Hat, Google, IBM, and a handful of others, and that has been true for well over a decade. The same pattern shows up across most infrastructure projects of any size. Volunteer contribution is real, yet paid engineers carry the load.

People who spend forty hours a week inside a company's engineering process bring that process with them. Review turns into an approval chain. Roadmaps get set by a small group. Release management belongs to specific people on a schedule. Disputes go up to whoever is supposed to end them. Nobody votes any of that in. It arrives as professional habit, because it is the only way of coordinating software most working engineers have ever practiced.

Core's contributors are largely employed or funded by a small number of organizations. The governance system that emerged at Core is what happens when people trained in corporate engineering coordinate a project and nobody is asked to design those rules on purpose.

The usual worry about concentrated funding is that donors pressure specific outcomes. That takes intent, coordination, and a visible fight. The quieter path runs all the time. People paid by companies bring their company's way of working with them, with no one telling them to.

Replacing the current contributors accomplishes nothing on its own. A different set of people, from the same professional background and funded the same way, would rebuild the same structure within a few years. The structure comes from where those people were trained, not from who they are. The social-layer version of that argument is *The Social Layer Is the Attack Surface*.

If paid engineers really import the hierarchy, a project staffed from different backgrounds and funded differently should not reconcentrate merge authority the same way. If it still does, the workplace-habit story is wrong, and hierarchy is coming from something more basic about coordinating software.

Not all of Core's structure is imported habit. Some of it was chosen on purpose. Merge access tightened after the inflation bug for stated reasons. Review culture hardened after the blocksize war as a considered response to what that conflict cost. Reluctance to touch consensus lightly is a position people hold out loud. Attributing all of it to unexamined corporate reflex would be too clean. Habit set the default, and deliberate choice then reinforced it.

The merge set also decides which changes count as consensus changes, which is how a proposal gets labeled too dangerous to merge. If that label is doing real work, it should show up in what shipped and what sat. That record is *What Bitcoin's Stalled Proposals Tell You*.

Bitcoin produces agreement, not a product

Bitcoin's coordination problem is agreement among peers who do not trust each other. Satoshi solved that for the chain. A bad block dies at every node that checks it. Linux produces a kernel people download. Bitcoin produces agreement about which chain is valid.

That check is only as good as the program doing it. The program is written by a small group. Inside that program the consensus rules are 10,122 code lines, 7% of the implementation, measured on Bitcoin Core 32.1 in *The Rules Are 7% of the Program*. The Commons white paper puts the set of people who can do consensus-critical Bitcoin work in the dozens to a few hundred worldwide. Miners and node operators are not in that set. They download a release, confirm it talks to their peers, and run it. They are trusting the maintainers and developers to have done the review they will not do themselves.

"Don't trust, verify" describes what a node does with a block, not what anyone does with a release. Your node checks every block against the rules. Almost nobody checks the software that holds those rules.

The trustless layer sits on a trusted one. The same gap, applied to Bitcoin Core, is *Don't Trust, Verify*.

The people who write that software are identifiable, and identifiable people can be pressured. If they fold, the software layer folds, whatever the cryptography says. A second client does not answer that by adding more people willing to say no. It answers it by giving operators somewhere to go. Refusing a release is only a refusal when there is other software to run, and without that software it is a delay.

Writing the code will never be as widely spread as checking blocks. Most people who run a node will never read it. That gap is what gives the authors more power than the operators, and it is why "the operators will catch it" is not a check. The gap is real and is not going away.

Refusing a release only delays the next download

A node can refuse a release. Other clients exist. btcd keeps diverging. libbitcoin exists and the ecosystem cannot use it without being rebuilt. A Core-derived fork that stays on Bitcoin is still mostly Core's code, so it is not a destination.

You can pin an old Core version for a while, and you cannot pin it forever. Unmaintained node software rots. Bugs pile up, the network moves, and eventually you take the next download or you fall behind. The people who write the default still govern, because refusal only delays the next download.

After the OP_RETURN relay change merged, Bitcoin Knots went from about 4.7% of reachable nodes at the end of April 2025 to a peak of 25.45% on September 14, 2025, and stayed above 19% through that stretch, according to Coin Dance. That was a policy protest on the same codebase, still one client. It did not give anyone a place to stay. Luke's Knots later left Bitcoin on a BLAKE2b proof-of-work hard fork. *Making Core Irrelevant* treats the reachable-node split as demand for a policy alternative, not as a break in the implementation monoculture.

Running different defaults is cheap compared to risking a chain split. Miners can refuse an upgrade, so a consensus change can be blocked. That veto is rarely used. The cost of refusing a default falls on the operator. The benefit of setting it accrues to whoever sets it. Nobody has to capture consensus enforcement to govern this way.

Undeclared concentration in the merge log

The stated model is rough consensus among a broad contributor base. In the merge log, a handful of accounts do most of the merges. Bitcoin Core is not formally a benevolent dictatorship and has never claimed to be, yet how it actually merges looks like a dictatorship with the name taken off, which is worse, because undeclared concentration is harder to see and harder to contest.

Bitcoin Core already posts ACKs on GitHub, which is not the same as the decision forming there. An ACK is a comment on a pull request. Which pull requests are going to merge is decided off GitHub, on IRC, in closed CoreDev sessions, and in Signal groups. Merge is a permission held by about five accounts on bitcoin/bitcoin. Reviewers do not have it, and one of those five ships the code.

Concentration of merge rights on its own still proves nothing. Any mature project restricts commit access, and Linux would look just as lopsided on the same measure. A high concentration of who may merge could just mean they are limiting who can push to a codebase where a bad merge could print coins. That would be a defense if the scarce mergers were also doing the review, and they are not.

Review activity Gini sits near 0.92, while merge-authority Gini is lower because the merger set is tiny, not because power is spread. Review is spread out relative to merges, and merge authority is not. Authorship Gini sits near 0.85. Brink's 2025 Engineering Impact Report published that in calendar year 2025 more than half of all merges flowed through a single individual funded by one grant organization. The Bitcoin Governance Research dataset puts the 2022+ window at 50.2% top-1 and 82.2% top-3. Self-merge among maintainer-authored merges is 25.5%, meaning the same people proposing changes were approving them on a quarter of those landings.

That dataset covers seventeen years of Core history and counts comments, ACKs, and review submissions on GitHub. It does not count hours spent checking consensus-critical code or how deep each review went, so these figures describe who shows up in the record, not who understood the code best in any given week.

The bug usually cited with these numbers is CVE-2018-17144, the inflation bug that sat in Core for eighteen months. People use it to prove Core is incompetent, which it does not prove. It sat in production, was never exploited, and was patched quickly after disclosure. Patching it fast after disclosure is not the same as catching it.

It propagated to Knots, ABC, and Unlimited, because those clients descend from Core's codebase. Listing those clients as independent checks is a mistake when they share a lineage. The case study is *Governance Paralysis Was The Victory*.

What the CVE does show is that implementations copied from a common codebase inherit that codebase's blind spots, which is the argument for building from a specification instead of observed behavior. An implementation written against a stated standard does not inherit those bugs by copying Core's source. It can still ship its own.

Three things that are fine for ordinary software are a problem in the process that writes Bitcoin's defaults. Decisions about what ships cannot be reversed except by whoever made them. Authority lives in the merge set without a procedure, so when that set leaves you have a crisis, and there is nothing to transfer except the habit of being listened to. Legal, financial, regulatory, or personal leverage only needs that handful.

Succession needs agreement that does not exist here

Every vertical project eventually loses its founder. Python is the usual counterexample. Guido stepped back, the community designed a steering council, and the language came through it intact, with no cryptographic governance, no formal specification, and no distributed signing authority. What carried it through was that people still wanted to work together, and that they had time. There was enough shared purpose to design a governance system after Guido left, and nothing catastrophic happened while they were designing it. Users just kept running the old interpreter and waited.

If Bitcoin's maintainers disappeared tomorrow, existing nodes would keep running. Python users can do the same thing. The difference is what cannot wait. CPython can sit unchanged while people design a council. Bitcoin still needs someone who can patch a bug that prints coins, and someone still picks what new operators download. You cannot put those jobs on hold until a new council exists.

Bitcoin also does not have the shared purpose Python had. The blocksize war was the test of whether it could resolve a fundamental disagreement through social process under pressure, and it could not do so without years of conflict and permanent factional damage. A succession plan that needs the community to agree under pressure has already been tested here, and it failed, which is why Core still cannot process contested work.

Taproot is the usual reply, because that change did ship and activate, yet it is the wrong comparison. Taproot was not the lead maintainer disappearing, and it was not a fight on the scale of the blocksize war. That it shipped says Bitcoin can activate a change when the disagreement is smaller, not that the community can design a governance system after the people in charge are gone.

Which thing gets forked

The blocksize war ended in a fork. Bitcoin Cash left, Bitcoin kept the ticker and the market. Some people say a fork is how this ecosystem keeps developers honest. The blocksize war was that fork. You cannot treat it as both the check working and proof that social process failed.

What matters is which thing gets forked. Bitcoin Cash forked the chain, which meant splitting the ledger, the hashrate, the economic network, and the community, permanently and by design. The cost of that exit was so high that it scares people off without keeping anyone honest. Nobody threatens it credibly, because the threat costs the threatener nearly everything they were trying to protect.

A governance fork is a different operation. Two implementations built against the same specification remain consensus-compatible, which means a disagreement about how a project is run can be resolved by one group building differently without anyone leaving the money. The chain, the ledger, and the economic network do not split. What splits is a development process, and a development process can afford to split. The chain-fork trap is *The Social Layer Is the Attack Surface, §VII*.

What a horizontal development layer looks like

Vote weight comes from work recorded in public, and a title does not confer it.

The Bitcoin Commons white paper sets the bar for a merge with two questions at once. One is where the change lives, in the consensus specification, the protocol, the node, or an optional module. The other is what kind of change it is, a routine fix, a feature, something that touches consensus validation, an emergency patch, or a change to the governance rules themselves. When both apply, you take the higher signature count and the longer wait. A bug fix in the consensus layer still needs the consensus-layer bar, while a feature in an optional module does not.

Routine maintenance clears on fewer signatures and a short wait. Consensus-adjacent changes need near-unanimity and a delay measured in months. Changes to the governance rules themselves need more than that. A contributor cannot approve their own merge, and even a repository administrator cannot bypass the signature check.

The patch that cannot wait runs through a declared emergency in three classes. Emergency keyholders declare a class, and that declaration changes the merge rules until a clock expires. Network-threatening work gets the fastest path, including no review wait while the declaration is live. Less severe classes keep longer waits and higher bars. Declaring an emergency is a different act from tagging a pull request as emergency work. After the clock, a written post-mortem is required and ordinary rules return. Nobody keeps a low bar because they might need one later.

Optional features load as process-isolated modules outside the money rules. Lightning, mining interfaces, and similar tools can compete without anyone treating them as a consensus change. The base node still validates blocks against the specification, so a feature fight does not become a fight about the money.

A further ladder, vote weight on the crates you have actually shipped, is specified as a complement to that merge policy, and it is not what is enforced today.

Every action is cryptographically signed and publicly auditable once that authorization is switched on. Power is visible because anyone can check the record, not because people are supposed to behave. The software layer cannot make every contributor equal the way every node is equal, and it can make the record checkable, which is what matters for who may change the code.

The missing specification is doing the work

None of that holds without a written specification. Fights still go to whoever has been around longest, because there is nothing independent to measure a judgment against, so authority reconcentrates no matter what a governance document says. Core cannot be repaired by redistributing merge rights, and the missing specification is what keeps the monopoly in place, because that missing document is doing work. The longer write-up is *Who Controls Bitcoin*.

A written specification gives contributors something to be right about that is not a person's opinion, and writing it is the hard part, because Bitcoin's live rules include serialization quirks, historical bugs the network accepted, and a policy and consensus boundary that is blurrier in practice than on paper. A specification that matches mainnet has to encode all of it, including the parts nobody would design on purpose, which makes it a large contested document that nobody writes in a week.

With a specification, fights become what the document says. Formal verification and machine-checkable proofs shrink the room to argue about what the specification means.

It is also what makes exit cheap enough to be credible. A formally specified implementation can be forked against the specification without reverse-engineering Core. A governance ruleset can be exported as a signed package and replaced without splitting the ledger, which is what makes a fork usable. Cheap exit keeps people honest in ways internal process cannot. The sequencing of that exit is *Making Core Irrelevant*.

The governance system described is Bitcoin Commons, the specification is the Orange Paper, and the implementation is BLVM.

Where this design fails

The crate ladder, if it ever ships, can let a shallow contributor outvote the person who wrote the subsystem. A single merged pull request can put someone on questions they cannot evaluate. Someone who has shown they understand the architecture still has to gate who votes. That limit is real. Below it, more votes make worse decisions.

The merge policy specified today has a different failure, because the people who hold the keys can collude. The keys are supposed to sit in different places for that reason, and a governance ruleset has to be forkable without splitting the money.

Distribution is slow when you need a same-day security patch. Ordinary merge rules should stay slow. The specified answer is a declared emergency with a clock, which is the substitute for a person who can merge anything.

If one implementation still defines the rules everyone else must copy, distributed governance is theater. A specification plus full-chain tests is what turns "we match Core" from a claim into a pass/fail check.

Commons has not run this merge policy at scale. Cryptographic merge authorization is designed and stays off until security review, key management, and community validation. Gates and declared emergencies are rules on a public record, and a person constrained only by custom is what they replace. That should break differently under pressure, and it remains a prediction until someone runs it. The specification, the emergency path, and cheap exit have to exist because of those limits.

The two questions that decide it

Bitcoin does not get a pass because some of the people writing the code are skilled or well-intentioned. Failed monetary institutions had skilled, well-intentioned people too. What matters is whether the people with merge authority still decide what Bitcoin nodes run on ordinary work.

It also matters whether an inflation bug can be patched without giving that merge set the power to merge whatever they want. The declared class and the expiring clock are the answer to that second question, which is why the overnight patch does not require a throne.

A governance system that works only while the current merge set holds the chairs was built for Linux, and it does not belong on Bitcoin's node software.

Sources

- Bitcoin Governance Research. Merge/review concentration, Gini, self-merge
- *The Rules Are 7% of the Program*. Line count of Bitcoin Core 32.1: 10,122 rule lines inside 151,004 implementation lines
- Brink, Engineering Impact Report 2025. One-person merge share
- Coin Dance. Knots reachable-node share
- Bitcoin Commons white paper. Dual bars (where and what), optional modules, three-class emergency (declaration vs PR tag), governance-ruleset exit
- Orange Paper / Bitcoin Commons consensus spec
- Bitcoin Commons documentation. BLVM Specification Lock; cryptographic merge authorization designed, off

SECURESOVEREIGN

Markdown is the canonical source: https://secsov.com/articles/the-vertical-layer-problem/index.md

Formatted snapshot of https://secsov.com/articles/the-vertical-layer-problem

Josh · Secure Sovereign